Deliverable 1

Expected results:
• Above 10% accuracy for both FCNN and CNN ✅

Deliverables:
• Code ✅

• Two test-cases for each custom layer function

```
MAX POOL TEST CASES:
TEST 1: PASS. Micah's maxpool produces the same result as the official implementation.
TEST 2: PASS. Maybe his maxpool wasn't a fluke!
CONVOLUTIONAL LAYER TEST CASES:
TEST 1: PASS. With identical starting weights, the functions produce the same convolution.
TEST 2: PASS. Maybe the conv2d wasn't a fluke either!
```

The actual implementation for the test cases is written in the code. However, I will give a high level overview of how I went about testing each function.

For verifying the validity of both the max pooling and convolution functions, I just grabbed the first two images from the train dataset to use as a test-case, as the images would both be different and serve as two different test cases. The idea is if the output of both of the output tensors are the same for the original pytorch implemented version of the function and my version of the function, then my functions should pass as successful implementations. This worked fine for my max pooling operation, but I ran into some complications for my convolution function, which I describe below:

At first, my tests were all failing until I realized that the randomly initialized weights between nn.Conv2d were of course different from the weights initialized form my own function. To work around this, I just used the F.conv2d function and created an output directly, using the exact same weights for both functions. To ensure complete fairness, I initialized the weights in the exact same way I would have within my own conv2d function, independent of my function (I initialized my weights to ensure independence.) I initialized with my conv2d function. This is how I make a fair comparison between my implementation and the official implementation.

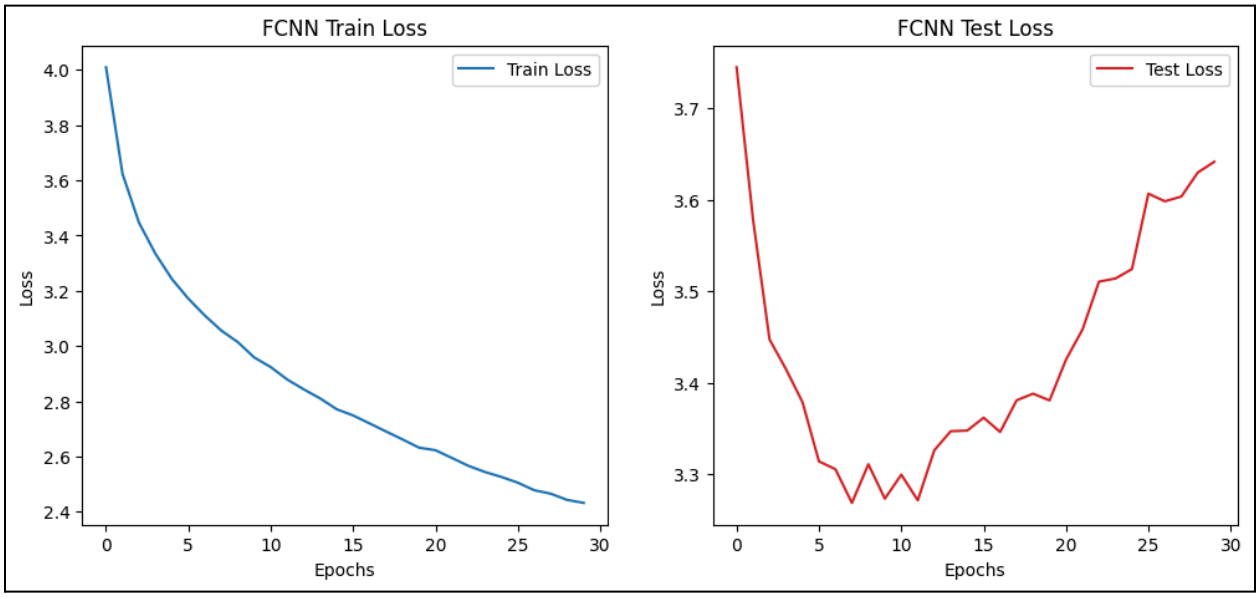
• **Max Accuracy and model architecture + hyper-parameters used for FCNN**

```
The max FCNN training accuracy was 0.3655690537084399.
The max FCNN testing accuracy was 0.23377786624203822.
```

```
Below is the FCNN model architecture
FCNN(
  (input): Linear(in_features=3072, out_features=500, bias=True)
  (act1): ReLU()
  (hidden): Linear(in_features=500, out_features=500, bias=True)
  (act2): ReLU()
  (output): Linear(in_features=500, out_features=100, bias=True)
)
```

```
Relevant hyperparameters:
Epochs: 30
Optimizer: Adam
Loss Function: CrossEntropyLoss
Learning Rate: 0.001
Activation Function: ReLU! I just wanted to go for simplicity
I also used a hidden layer of 500 neurons to make sure that the FCNN was capable of learning the CIFAR100 data to some extent.
The max FCNN training accuracy was 0.3655690537084399.
The max FCNN testing accuracy was 0.23377786624203822.
```

• **One pair of train and test losses plots for FCNN**



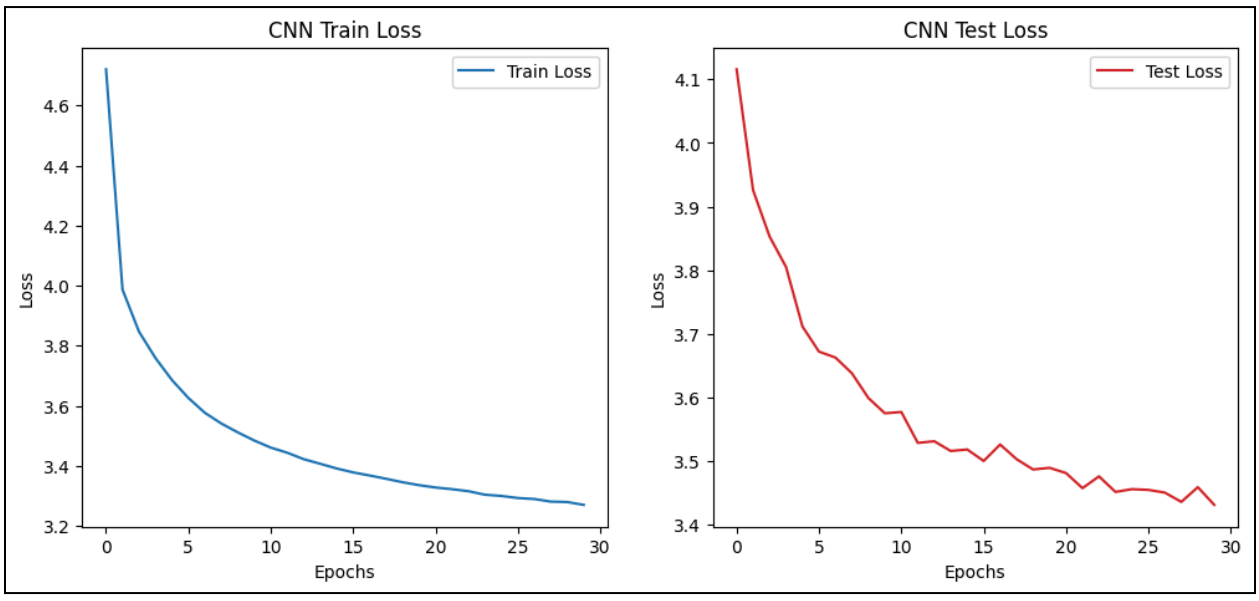
• **Max Accuracy and model architecture + hyper-parameters used for CNN**

```
The max CNN training accuracy was 0.22516384271099743.
The max CNN testing accuracy was 0.19944267515923567.
```

```
Below is the CNN model architecture
MyCNN(
  (conv1): MyConv2D()
  (act1): ReLU()
  (maxpool1): MyMaxPool2D()
  (conv2): MyConv2D()
  (act2): ReLU()
  (maxpool2): MyMaxPool2D()
  (linear): Linear(in_features=150, out_features=100, bias=True)
)
```

```
Relevant hyperparameters:
Epochs: 30
Optimizer: Adam
Loss Function: CrossEntropyLoss
Learning Rate: 0.001
Activation Function: ReLU! ReLU is the standard for CNNs.
I went for two convolutions because I want higher and lower levels of features to be learned.
```

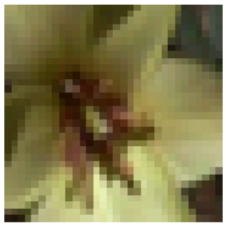
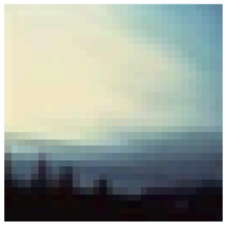
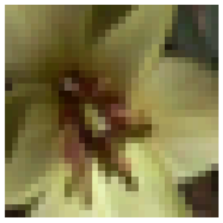
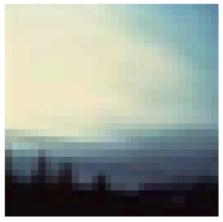
• **One pair of train and test losses plots for CNN**



• Explain all learning and insights from changes made to achieve highest possible accuracy

For better or for worse, I had a lot of learning and insights. For starters, I initially implemented the CNN functions correctly, making use of for loops (which is the naive approach to convolutions), and I ended up spending hours trying to run the model on vscode and google colab, only to decide that architecture was too inefficient to learn fast enough, meaning I had to scrap my current implementation of the functions. My scrapping of the old implementation of the functions ended up leading me to achieve a much higher possible accuracy for my CNN, as I was now able to train my CNN for more epochs with ease. For the record, it was talking around the scale of a single minute to load a single batch when I was using my naive CNN function implementation; for reference, this would mean I would've had to train my network for 37 hours for just 20 epochs. After some research, I discovered that torch.unfold and making use of the precomputation of the sliding windows and patches is the way to go, being much more efficient than the naive method that I discussed before. With this new implementation, my CNN was now breezing through not just batches but also entire epochs. 30 epochs were done in mere minutes, which was an INSANE improvement. In terms of improving the accuracy itself, for some reason, I forgot to include ReLU activation functions between the convolutional layer and the max pool layer. I definitely think adding this nonlinearity must have really helped boost the CNN model classification accuracy, although I was not sure, since I did not compare these two models after I achieved a decent accuracy for my CNN.

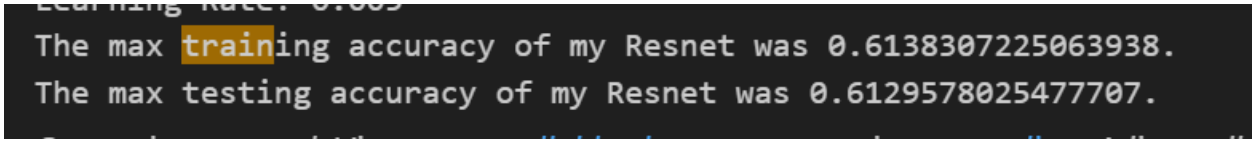
My FCNN was very obedient. It listened to me in almost the first pass through implementing it. I think part of the reason why is that I didn't implement the FCNN in any way that's aberrant to how you would normally implement a fully connected neural network. I used ReLU activation functions, which are quite standard. However, I made a minor change to increase the number of hidden neurons in the hidden layer (I think from 100 to 500), and this drastically improved my FCNN performance. I suspect this gave my network a greater capacity to learn. While this probably led the FCNN to overfit (as you can see from the test loss graph), I was still happy with my results, as I achieved higher than 10% accuracy and I already know that FCNN is the naive approach for image classification tasks.

Visualization of FCNN predictions	Visualization of CNN predictions
FCNN Predictions Actual Class:tulip; Predicted Class: bee  Actual Class:camel; Predicted Class: crocodile  Actual Class:butterfly; Predicted Class: beetle  Actual Class:cloud; Predicted Class: cloud  Actual Class:apple; Predicted Class: bowl 	CNN Predictions Actual Class:tulip; Predicted Class: tiger  Actual Class:camel; Predicted Class: porcupine  Actual Class:butterfly; Predicted Class: bicycle  Actual Class:cloud; Predicted Class: skyscraper  Actual Class:apple; Predicted Class: pear 

Deliverable 2

Expected results:

• Test and Train Accuracy of 60% or higher for full credit for your best model on CIFAR-100 dataset ✅



WOHOOOOOOOOOOOOOO!!!!!!!!!!!!!!!!!!!!!!!!!!!!

• Visualization of results and number of correct predictions visually out of 10. ✅
-> see later in the document! I have my visualizations below the learning and insights question!!!

Deliverables:

• Code ✅

• Max Accuracy and model architecture + hyper-parameters used

```
The max training accuracy of my Resnet was 0.6138307225063938.
The max testing accuracy of my Resnet was 0.6129578025477707.
```

```
Below is the Resnet model architecture
MyResnet(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (batch_norm): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (act1): ReLU()
  (maxpool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (residual_blocks_1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Conv2d(32, 64, kernel_size=(1, 1), stride=(1, 1))
    )
```

```
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (residual_blocks_2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(96, 96, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Conv2d(64, 96, kernel_size=(1, 1), stride=(2, 2))
    )
    (1): BasicBlock(
      (conv1): Conv2d(96, 96, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(96, 96, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
```

```
(residual_blocks_3): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(96, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(96, 96, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (downsample): Conv2d(96, 96, kernel_size=(1, 1), stride=(2, 2))
  )
  (1): BasicBlock(
    (conv1): Conv2d(96, 96, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(96, 96, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
(residual_blocks_4): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(96, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (downsample): Conv2d(96, 128, kernel_size=(1, 1), stride=(2, 2))
  )
```

```
  )
  (1): BasicBlock(
    (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
(AveragePooling): AdaptiveAvgPool2d(output_size=1)
(linear1): Linear(in_features=128, out_features=50, bias=True)
(dropout): Dropout(p=0.3, inplace=False)
(linear3): Linear(in_features=50, out_features=100, bias=True)
)
```

hyperparameters:

```
Relevant hyperparameters:
Epochs (this was the best epoch, where I stopped training): 46
Optimizer: SGD
Loss Function: CrossEntropyLoss
Learning Rate: 0.005
```

more on next page:

optimizer = SGD

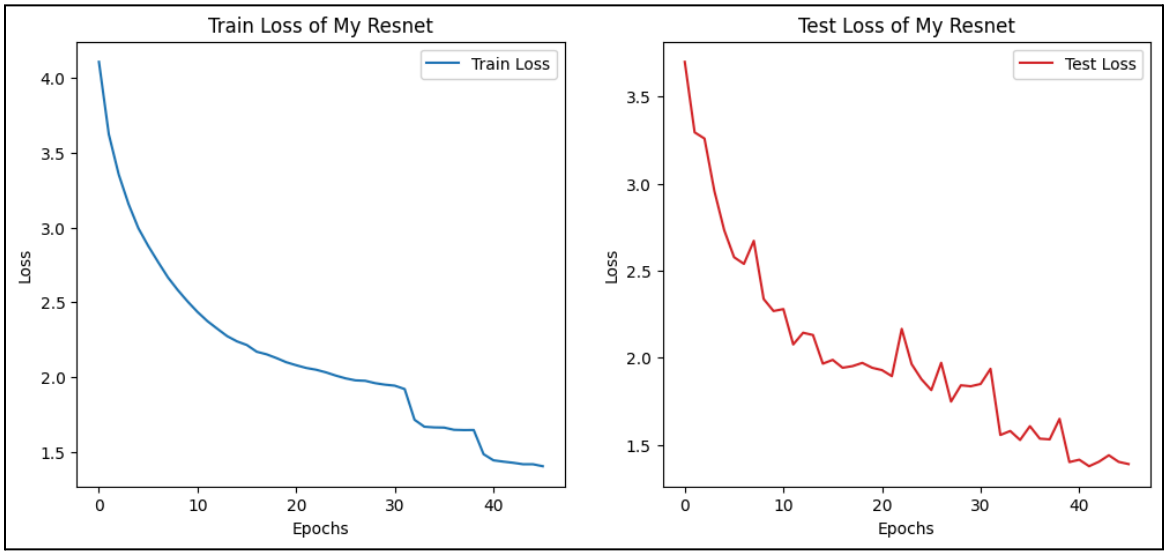
- **momentum=0.9**
- **weight_decay=2e-3**
  - **I used a high weight decay to encourage model L2 regularization. The model was overfitting like crazy, so this was the most logical step for me.**
- **nesterov=True**
  - **This helps SGD converge beyond what it's normally able to by adjusting from looking ahead rather than the current state of the model**

learning rate scheduler = ReduceLROnPlateau

- *I discovered this after experimenting with CosineAnnealingLR and it helped significantly. It has an advantage in that it adjusts the LR based on the validation loss, which was the main limiting factor for me, as my train and test sets kept having such a huge difference in accuracy.*
- **mode = 'min'**
  - **since I pass test loss into this metric, it is what the scheduler is trying to minimize**
- **factor = 0.5**
  - **reduces the learning rate by a factor of 2 to allow the test accuracy to catch up**
- **patience = 3**
  - **this is the trigger condition for when the learning rate is shifted, how long the scheduler waits for no improvement in the test loss to occur.**

I also set the number of initial filters to 32 and the number I would use to include the number of neurons in the hidden layer of the fully connected linear layers at the end to 100.

• One pair of train and test losses plots



• Explain all learning and insights from changes made to achieve highest possible accuracy

I grew so much from this question, it's insane. I honestly feel like I doubled my capabilities as a deep learning engineer, and that's not even an overstatement. I actually invested a lot of time in learning how residual blocks work, the heart of resnet18 because I figured that if I could implement my own residual blocks (which I did via Basic Block), then I could create a pipeline that works best for CIFAR100.

After some research, I realized that while resnet18 is awesome for large images, it's actually kind of clunky and aggressive. For example, the first layer of resnet18 starts with a kernel size of (7x7) and its whole architecture involves aggressive downsampling since it's trained on insanely large images compared to CIFAR100. With this in mind, I went and did something much more practical. Since we can import pretrained models without any penalty, I went ahead and imported the residual blocks from pretrained models without any penalty. I basically made use of these blocks by creating my own architecture that is robust to the small input size of CIFAR100. My main plan was to be very kind and gentle to the small images unlike the aggressive resnet18 and avoid downsampling by a lot, just gradually accumulating important features until the adaptive pooling layer at the very end.

However, even with my custom made architecture, I struggled greatly to push the model past 60% accuracy for the test set specifically. I experimented with preprocessing, normalizing both the train and test sets. To combat overfitting, I tried implementing dropout in my final linear layers, and I learned that the dropout actually has a profound ability to generalize, just like adjusting the weight decay of the optimizer. Speaking of optimizers, at first I tried using Adam. That didn't work, so I went to SGD using cosine annealing as a scheduler. This was really effective in the long run. However, I needed something that was even more efficient but also responsive to the test loss. That was when I discovered ReduceLROnPlateau, which was honestly incredibly helpful in helping my test accuracy to catch up to my train accuracy. I tried implementing multiple different linear layers at the end, trying 1, 2, and 3, finally settling on 2. I tried many learning rates, but settled on using SGD with a really low learning rate of 0.005 to start.

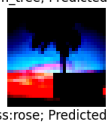
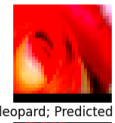
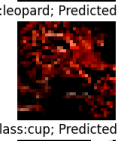
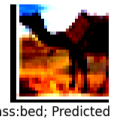
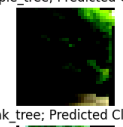
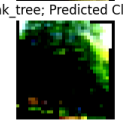
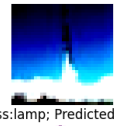
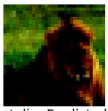
I also made extensive use of data augmentation to help generalization. My augmentations included but were not limited to:

- Random cropping with padding (keeping size the same but obscuring part of the image)
- Random horizontal flipping
- Random Erasing
- Random rotation (-4 to 4 degrees)
- Color jitter (adjusting brightness, contrast, saturation, and hue)

After all these adjustments, I finally started seeing consistent improvements, and it felt insanely rewarding to watch the model actually learn and generalize instead of just memorizing the training data. The fact that I reached above 60% accuracy for both the train and test data is insane to me before epoch 50. I didn't keep training the model, but I wonder how high it would've gone!

• **Show visualization results**
• Visualization of results and number of correct predictions visually out of 10.

LARGER IMAGES ON NEXT PAGE.

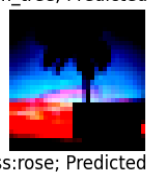
Performance of MyResnet on Train Data	Performance of MyResnet on TestData
9/10 Predictions from My Resnet on Train Data: 90.0000% Accuracy! Actual Class:snail; Predicted Class: snail  Actual Class:elephant; Predicted Class: elephant  Actual Class:palm_tree; Predicted Class: palm_tree  Actual Class:rose; Predicted Class: rose  Actual Class:leopard; Predicted Class: leopard  Actual Class:cup; Predicted Class: cup  Actual Class:camel; Predicted Class: camel  Actual Class:bed; Predicted Class: bed  Actual Class:maple_tree; Predicted Class: maple_tree  Actual Class:oak_tree; Predicted Class: maple_tree 	**9/10** Predictions from My Resnet on Test Data: 90.0000% Accuracy! Actual Class:sea; Predicted Class: bridge  Actual Class:skunk; Predicted Class: skunk  Actual Class:streetcar; Predicted Class: streetcar  Actual Class:rocket; Predicted Class: rocket  Actual Class:lamp; Predicted Class: lamp  Actual Class:lion; Predicted Class: lion  Actual Class:tulip; Predicted Class: tulip  Actual Class:wolf; Predicted Class: wolf  Actual Class:rose; Predicted Class: rose  Actual Class:orange; Predicted Class: orange 

Predictions from My Resnet on Train Data: 90.0000% Accuracy!

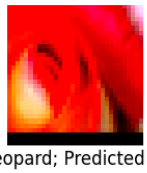
Actual Class:snail; Predicted Class: snail

Actual Class:elephant; Predicted Class: elephant

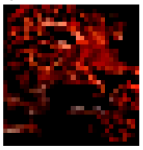
Actual Class:palm_tree; Predicted Class: palm_tree



Actual Class:rose; Predicted Class: rose



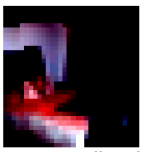
Actual Class:leopard; Predicted Class: leopard



Actual Class:cup; Predicted Class: cup

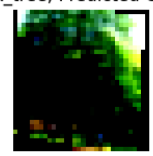
Actual Class:camel; Predicted Class: camel

Actual Class:bed; Predicted Class: bed



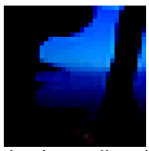
Actual Class:maple_tree; Predicted Class: maple_tree

Actual Class:oak_tree; Predicted Class: maple_tree



Predictions from My Resnet on Test Data: 90.0000% Accuracy!

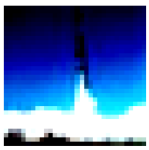
Actual Class:sea; Predicted Class: bridge



Actual Class:skunk; Predicted Class: skunk

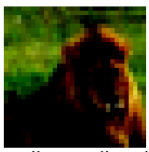
Actual Class:streetcar; Predicted Class: streetcar

Actual Class:rocket; Predicted Class: rocket

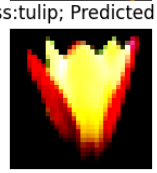


Actual Class:lamp; Predicted Class: lamp

Actual Class:lion; Predicted Class: lion



Actual Class:tulip; Predicted Class: tulip



Actual Class:wolf; Predicted Class: wolf

Actual Class:rose; Predicted Class: rose

Actual Class:orange; Predicted Class: orange

Deliverable 3

Explain your approach to making a detector using a classifier

I chose resnet18 as my classifier because it is fast and robust. I started by seeing how resnet18 would classify the image as a whole. Based on the highest probability classes, I proceeded by focusing on finding sub-images that matched the highest probability class. I was taking advantage of the parameters of the assignment here, as I ended up just needing to locate the probability of a tabby cat or redbone (which is a type of dog).

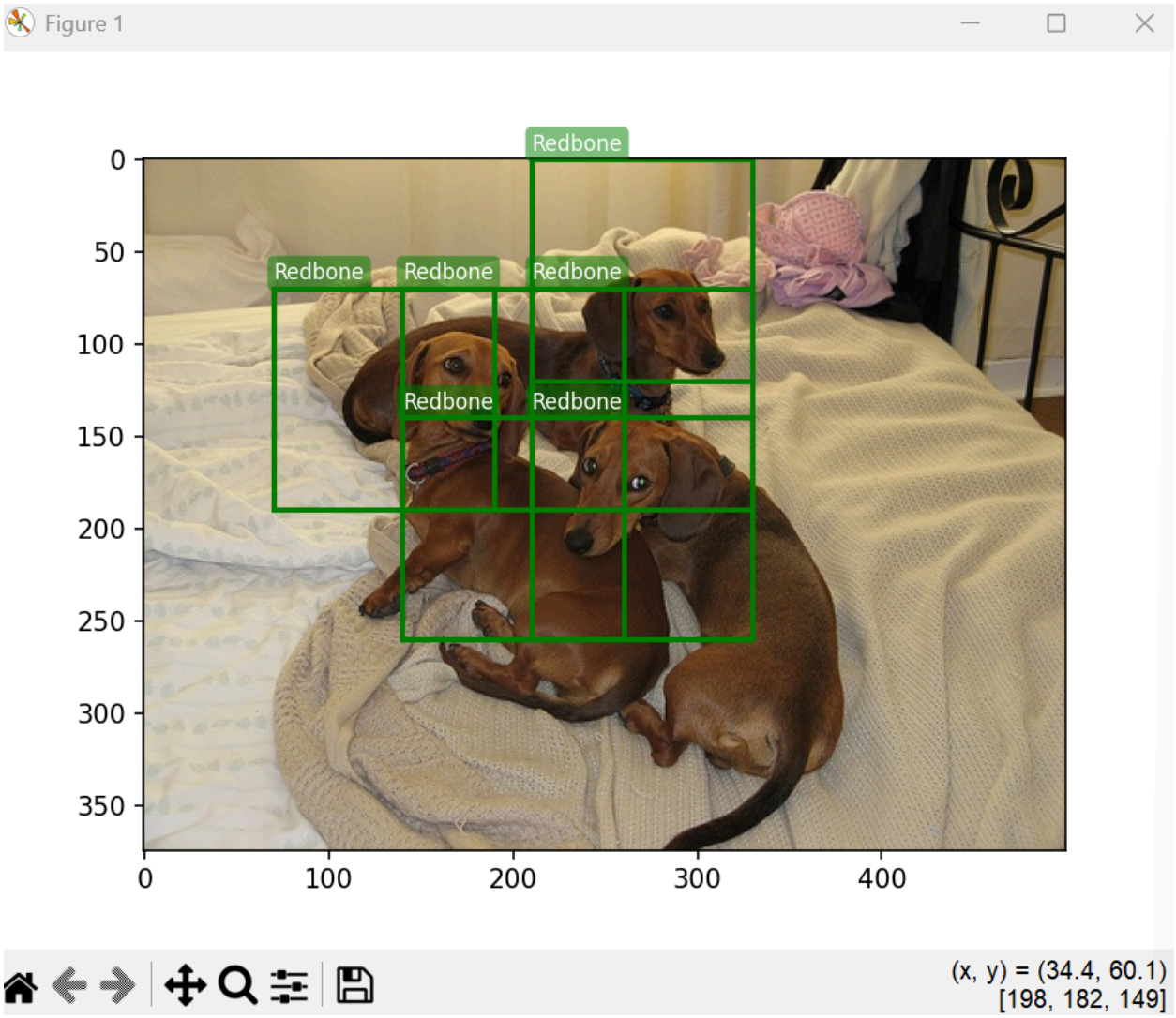
In terms of windowing, I used the trick from numpy, which I felt was the most straightforward way (compared to torch folding and unfolding. I realized that you can iterate through the resulting windows with a stride/certain step size, and I really took advantage of this, as modifying the step size led to different windows, which led to different probabilities of classification.

Once I had the architecture of being able to create varying window sizes that could slide across an image with varying step sizes, I felt that I was ready to start working on the actual detector aspect. In order to only show windows relevant to the cat or the dog, I created a threshold which does not include windows if the probability of them including a cat or dog is too low. To determine the accuracy of the detector, I verified the image myself, using the open source cv2 library to draw green bounding boxes. In order to create the best classifier which would lead to several bounding boxes in the general region of the cats and dogs, I had to play around with the window size, stride size, and threshold a bit, similar to how one would with a machine learning optimization problem. Importantly, I trained two different classifiers for both dogs and cats using different parameters because I felt that to obtain the best results, it made the most sense to use slightly different window sizes, as this would affect the resulting windows slid across the screen, along with the probabilities of a dog or cat respectively. While I ran out of time, I really wanted to implement an algorithm to merge the boxes based on which of the boxes exceeded a certain probability threshold. For all intersecting boxes that did exceed that threshold, I would choose the one with the highest probability and merge the other boxes to that box. However, I am happy with my images, as I still ended up with several bounding boxes in the general region of the cats and dogs. I have to admit, I was a little confused on whether I should be merging the bounding boxes or not, though. The results said that it was expected that several bounding boxes are supposed to be in the region of cats and dogs, but several often means more than five. Since there are only three animals in each image, I wasn't sure if merging was something required and needed if the only expected result listed was that there are several bounding boxes in the general region of the cats and dogs.

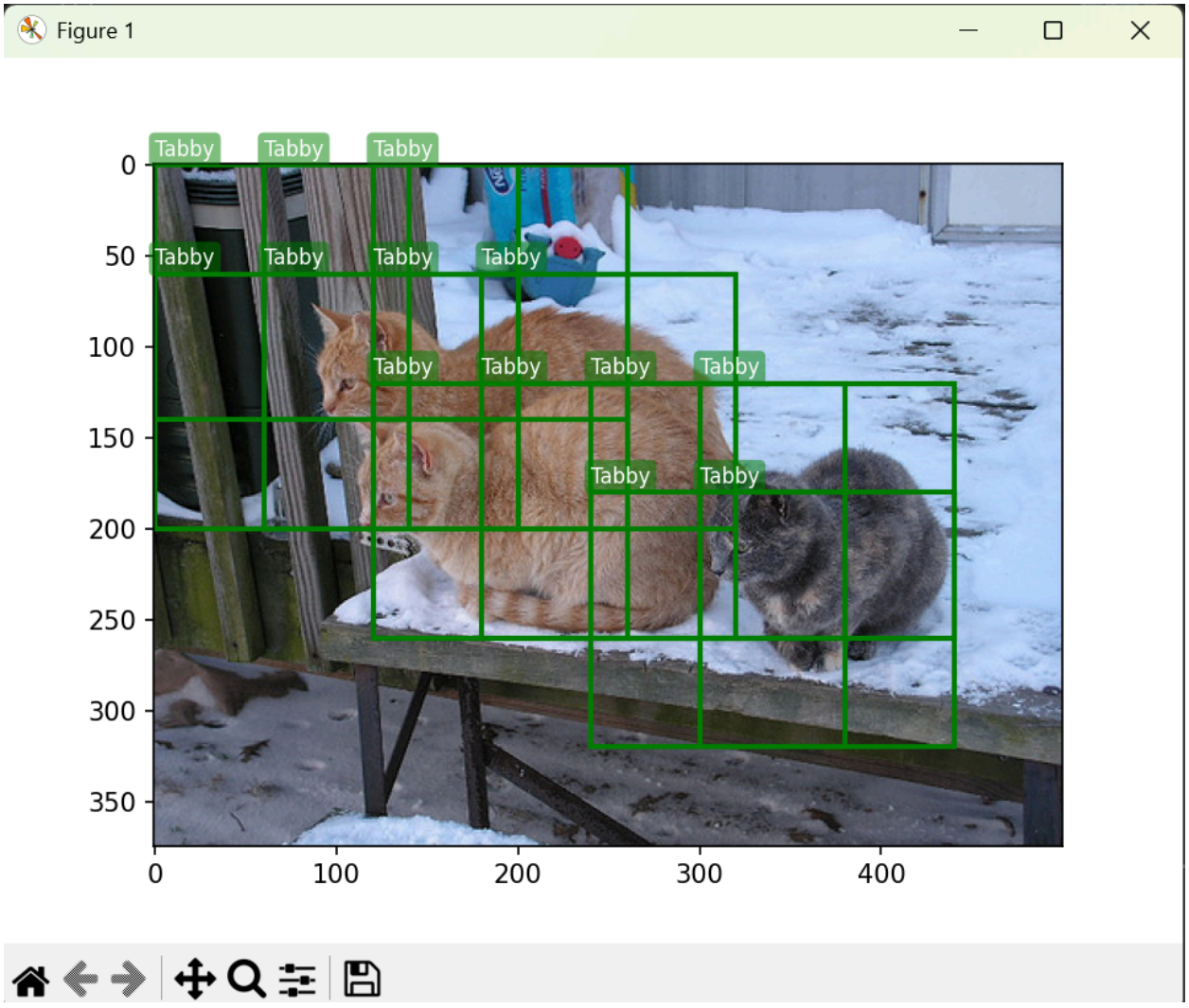
The cat and dog bounding box pictures are on the following page.

Bounding Box Images:

Dog



Cat



Deliverable 4

Deliverables
• Code ✅

• latency for YOLO

```
Across 100 samples,
Average YOLO Latency: 0.047 seconds!
```

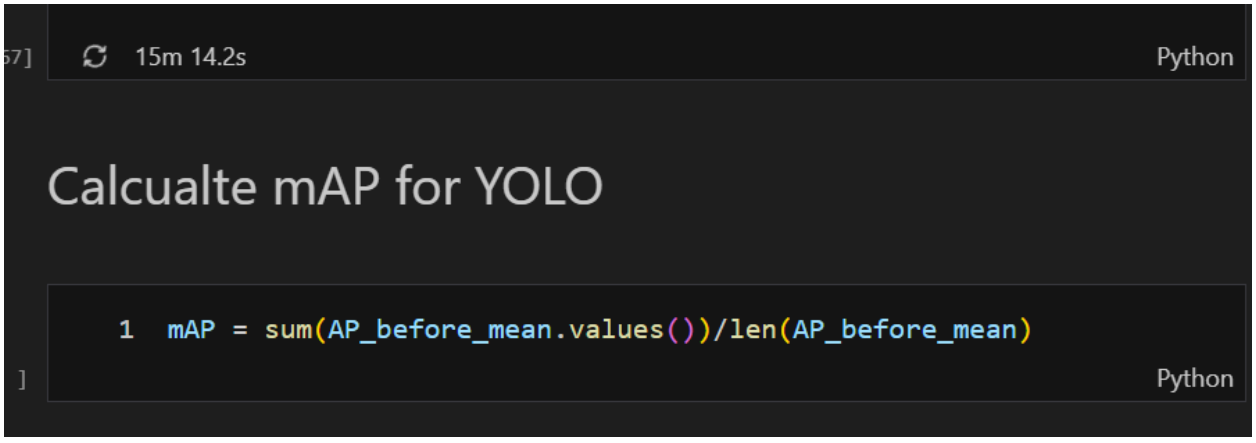
• latency for Faster-RCNN

```
Across 100 samples,
Average Faster_rcnn Latency: 0.062 seconds!
```

• mAP for YOLO and Faster-RCNN

Unfortunately, I failed to produce a valid mAP score for both YOLO and Faster-RCNN. However, I got insanely close to doing it with YOLO.

In fact, I had written out 90% of the code to calculate the mAP score, even finishing out the pipeline:



To be honest, the only reason I did not finish on time is I have been running into two main issues

- It takes a really long time to loop through every image in the coco dataset and collect the precision and recall across

    - This is because I didn't attempt to calculate the PR scorers while making use of the batches. I loaded the images one at a time because for me, this made it much easier to unpack what was going on and how to match between the ground truth annotations and the YOLO predictions.
- I was really struggling between the difference in YOLO and COCO class mappings. It felt like I was solving a jigsaw puzzle of another jigsaw puzzle. If this was more streamlined, I think it would have been easier overall to implement mAP.

I think where it stands now, I made a few mistakes in my computations, leading to my end product being too inefficient to be able to produce a result. However, I genuinely believe that I am not that far off from producing a valid mAP score for the YOLO model. I have most of the parts; they just need some care and debugging.

However, if I was able to get a mAP score for YOLO, I would expect it to be around 37.3, based on the official website and the model I downloaded:

Model	size (pixels)	mAP^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7

,

Moreover, if I was able to get a mAP score for Faster-RCNN, I would expect it to be around 22.8, based on the pytorch official website, performing decently worse than YOLO.